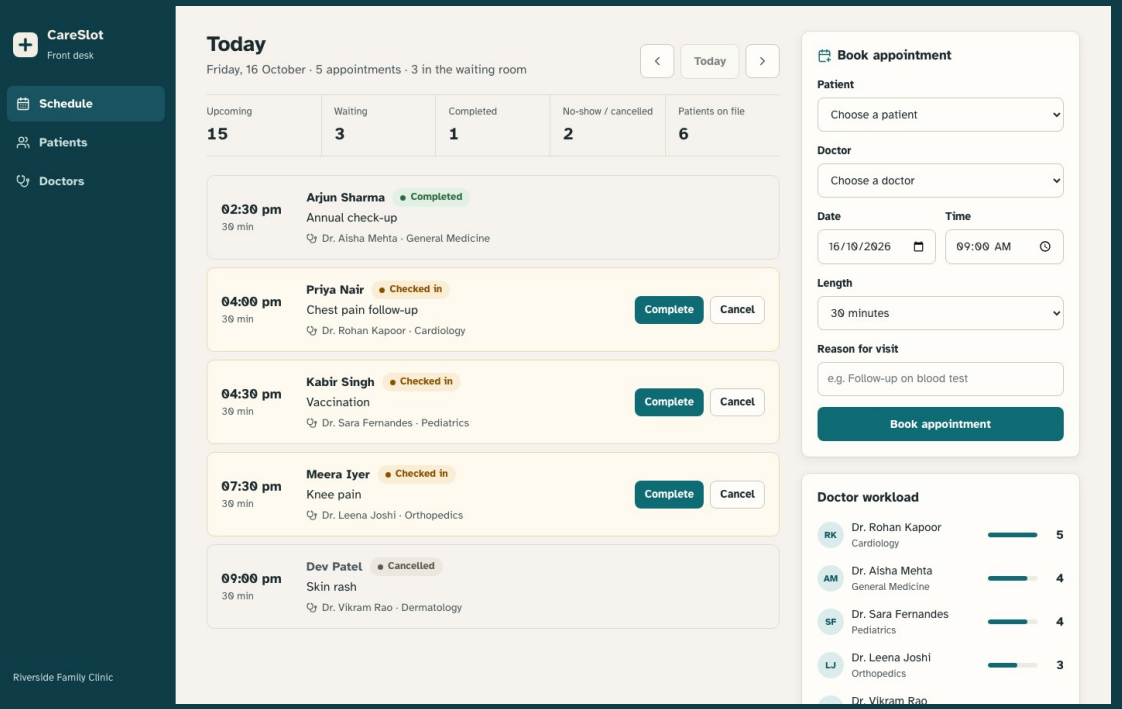
CareSlot

Clinic appointment scheduling, shipped from laptop to a monitored Kubernetes cluster



DevOps Final Capstone · Session 21
github.com/MadaraUchiha-tech/devsecops-python

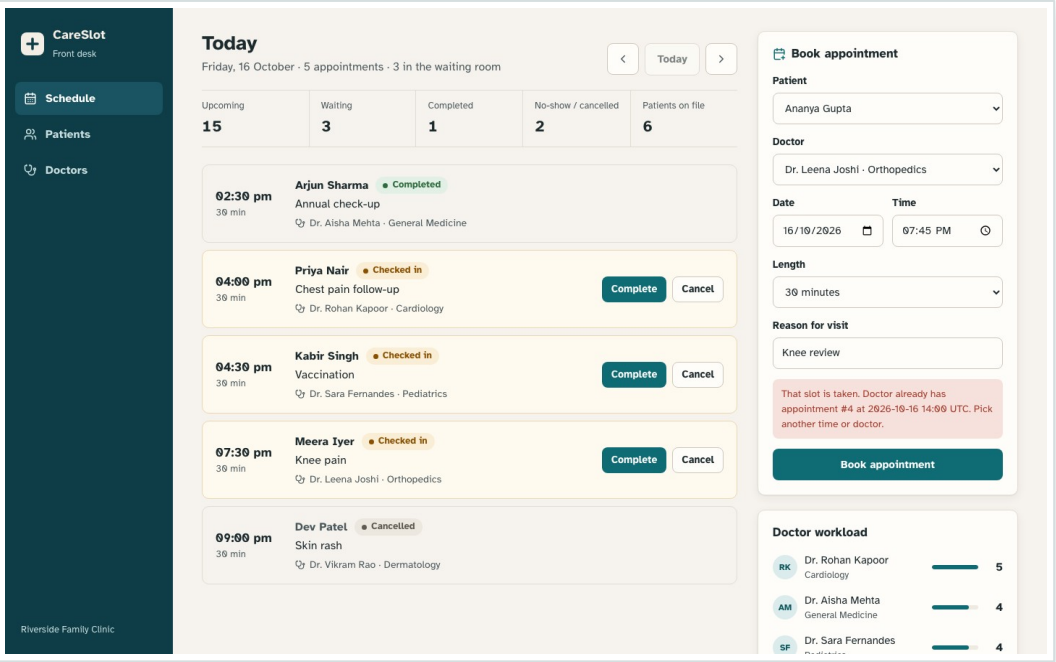
Front desks double-book doctors

Small clinics still schedule on paper or shared spreadsheets. CareSlot replaces that with one source of truth.

Double booking
Two receptionists book the same doctor at the same time. The API refuses overlapping slots with a 409.

No view of the waiting room
Check-in, completion, cancellations and no-shows are tracked as a status workflow.

Uneven doctor workload
A live workload panel shows upcoming visits per doctor; doctors can go off duty.



The booking form rejects a slot that overlaps an existing visit

The application: React, FastAPI, PostgreSQL

16

REST endpoints across doctors, patients, appointments

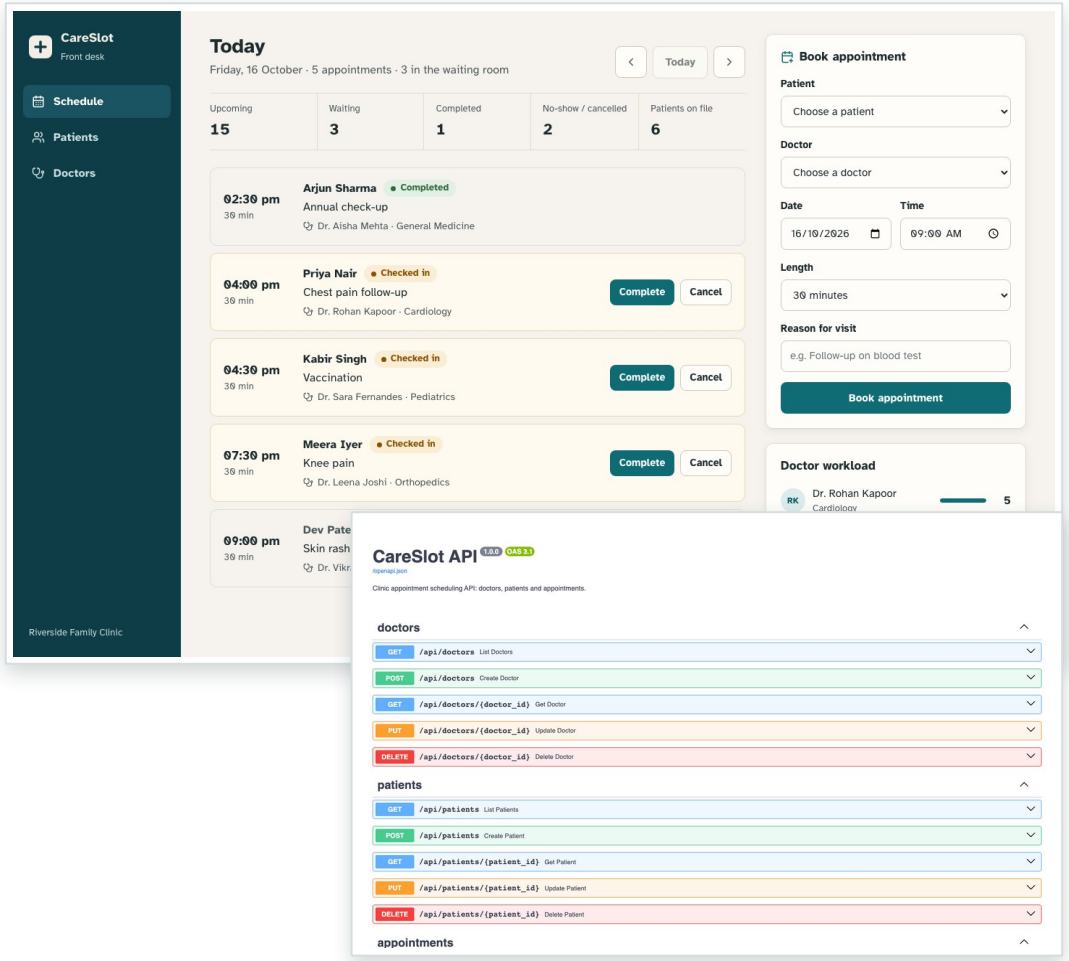
21

pytest cases, 94% line coverage

5

appointment states with enforced transitions

- SCHEDULED → CHECKED_IN → COMPLETED; cancel or mark no-show along the way
- Overlapping bookings for a doctor return 409 with the clashing visit
- Alembic owns the schema; demo data seeded behind a Postgres advisory lock
- /health (liveness), /ready (DB check), /metrics (Prometheus) for Kubernetes



Tech stack

Application
React 19 · Vite · FastAPI · SQLAlchemy · Alembic · Postgres 16

Quality
pytest · pytest-cov · Ruff lint + format

Containers
Multi-stage Docker · Compose · nginx-unprivileged · non-root

CI/CD
GitHub Actions · GHCR · SHA-tagged images · kind smoke test

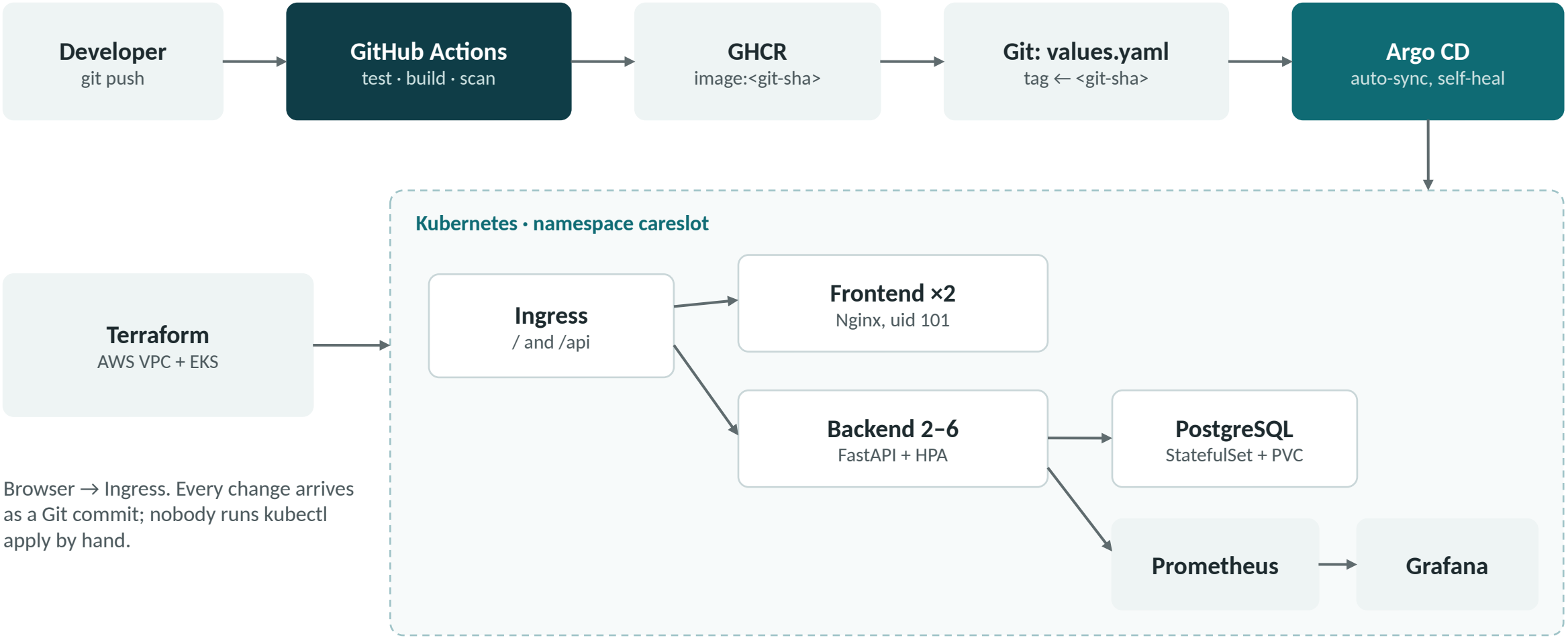
Security
Trivy image, secret and IaC scans · SARIF · Pod Security

Infrastructure
Terraform modules · AWS VPC · EKS · IAM · KMS

Kubernetes
Helm 3 · ingress-nginx · HPA · Argo CD (GitOps)

Observability
kube-prometheus-stack · ServiceMonitor · Grafana

Architecture: Git is the only way into the cluster



Browser → Ingress. Every change arrives as a Git commit; nobody runs kubectl apply by hand.

CI/CD: nothing reaches the registry untested or unscanned

1 Quality gates
Ruff + pytest · npm build · terraform validate · helm lint · Trivy secrets

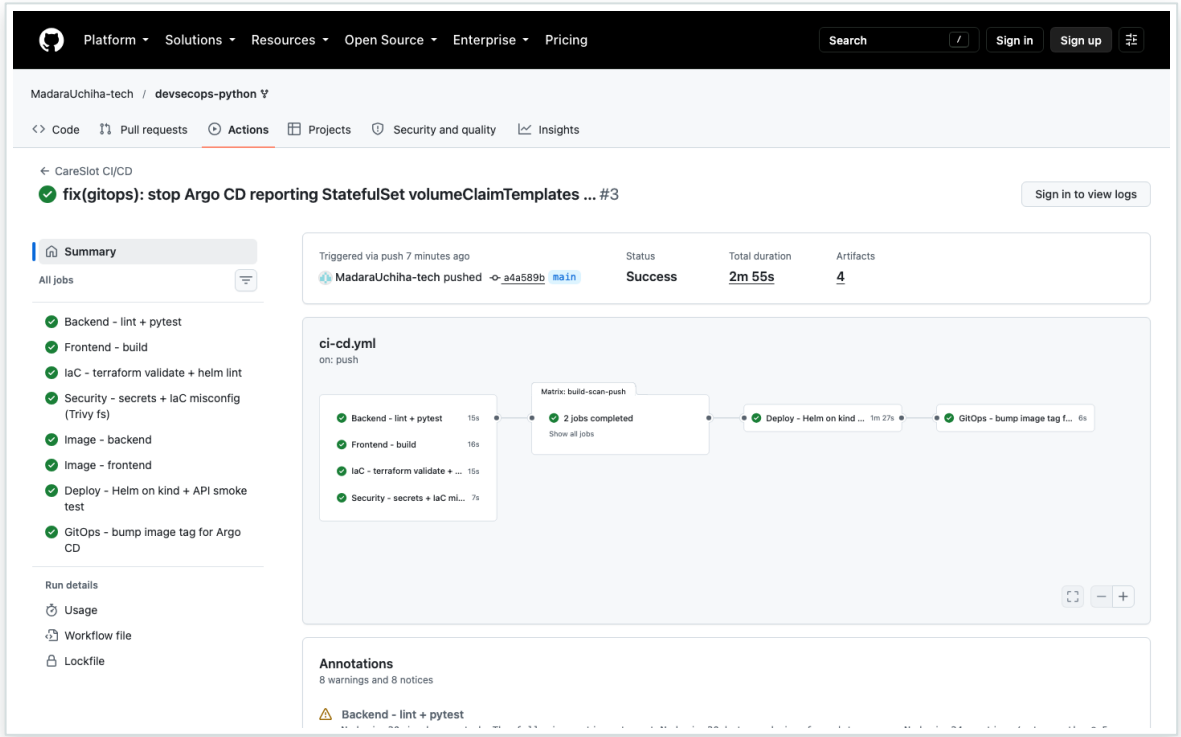
2 Build
Backend and frontend images, tagged with the commit SHA (never latest)

3 Trivy gate
Fail on any fixable HIGH/CRITICAL CVE; SARIF to GitHub Security

4 Push
Only scanned images go to ghcr.io/madarauchiha-tech/careslot-*

5 Smoke test
kind cluster + helm upgrade --install + book a real appointment

6 GitOps promote
CI commits the SHA into values.yaml; Argo CD rolls it out



Run #3: all 8 jobs green in 2m 55s · images in GHCR tagged a4a589b…

DevSecOps: Trivy blocked a real vulnerability

7

HIGH CVEs in the first backend image

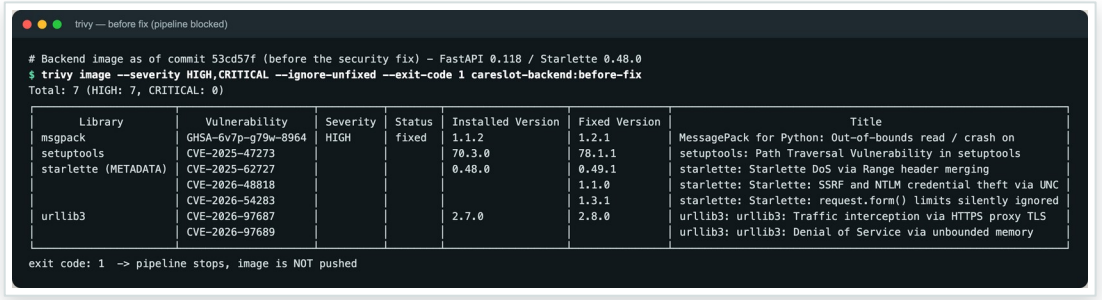
0

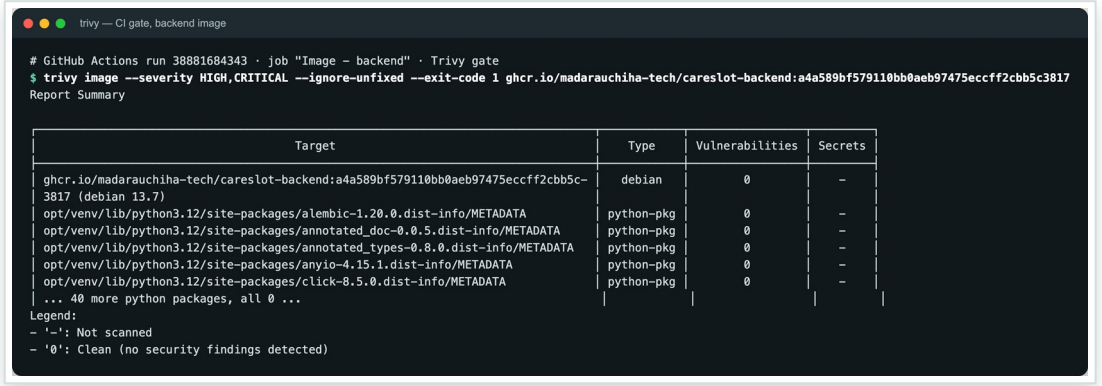
fixable HIGH/CRITICAL in both images now

CVE-2025-62727 · Starlette 0.48.0

A request with many overlapping Range headers made Starlette do quadratic work merging them: a few requests could pin a CPU (denial of service). Fixed in 0.49.1.

Fix: upgrade FastAPI/Starlette, and strip pip and setuptools (and their vendored urllib3) from the runtime image.





Also: Trivy secret scan (blocking) · IaC scan led to KMS-encrypted EKS secrets and a read-only Postgres rootfs · non-root pods, all capabilities dropped

Infrastructure as Code: VPC + EKS in Terraform

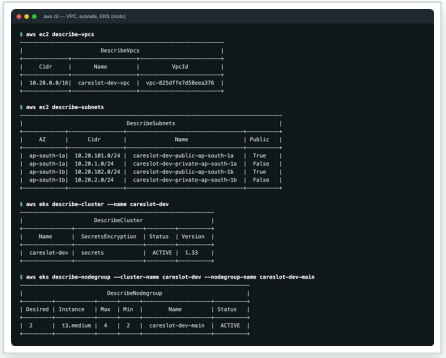
VPC 10.20.0.0/16 · ap-south-1

ap-south-1a	ap-south-1b
Public subnet 10.20.101.0/24 · NAT gateway	**Public subnet** 10.20.102.0/24 · ELB-tagged
Private subnet 10.20.1.0/24 · worker nodes	**Private subnet** 10.20.2.0/24 · worker nodes

EKS careslot-dev · Kubernetes 1.33
managed node group t3.medium ×2 (max 4) · KMS-encrypted secrets · audit logs

28 resources: plan → apply → destroy, all clean

- Two modules (vpc, eks), variables with validation, tfvars.example; no credentials in Git
- Run against moto, a local AWS API emulator with dummy credentials: the real lifecycle at zero cost
- Same code applies to a real account: terraform init / plan / apply / destroy



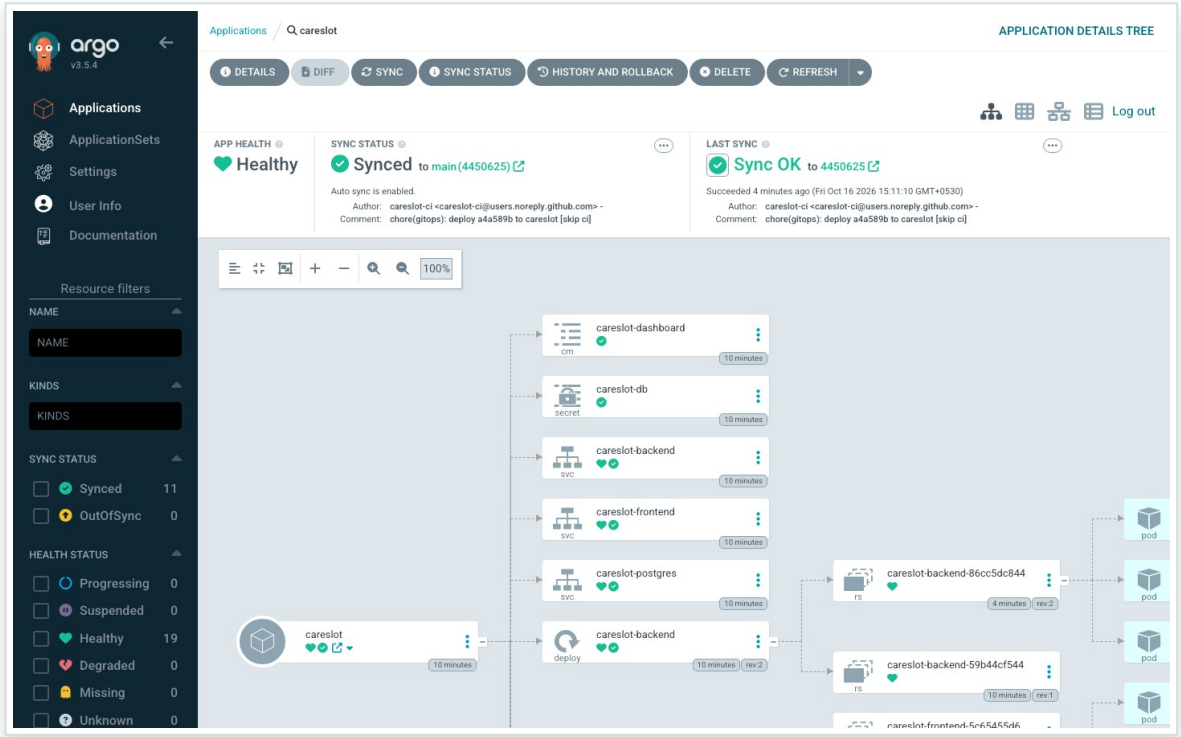
Kubernetes, Helm and GitOps with Argo CD

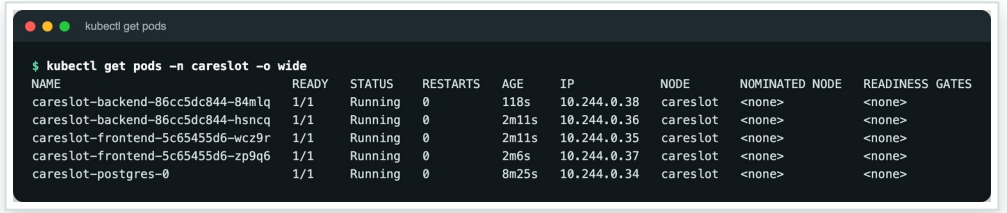
Helm chart
Deployments ×2, ClusterIP Services, Postgres StatefulSet, Secret, Ingress, HPA, ServiceMonitor, dashboard

Hardened pods
runAsNonRoot, read-only rootfs, caps dropped, Pod Security baseline, quotas

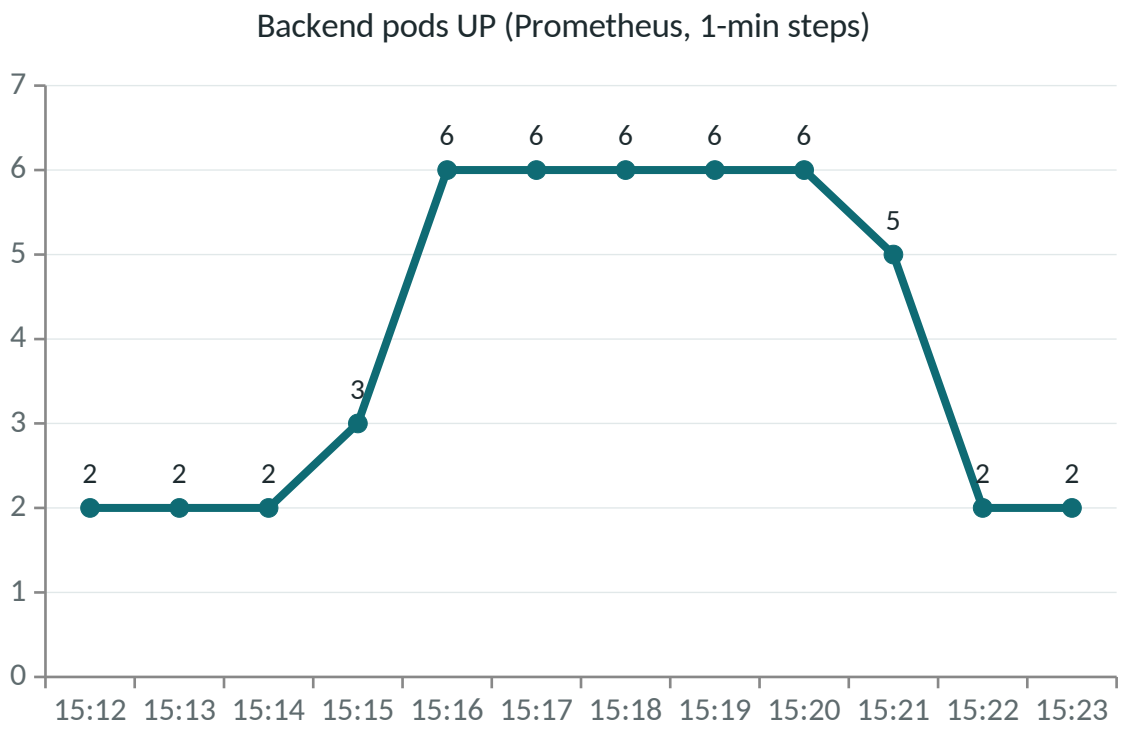
Argo CD
Auto-sync from main, prune, self-heal: a manual scale-down was reverted in seconds

Ingress
careslot.local: / → frontend, /api → FastAPI (ingress-nginx on minikube)





Autoscaling under load: 2 → 6 backend pods



Load test 15:14–15:18 at ~200 req/s (60 workers). CPU peaked at 462% of request vs a 60% target; HPA hit its max of 6, then scaled back to 2 after the 2-minute stabilisation window.

HPA scaling under load

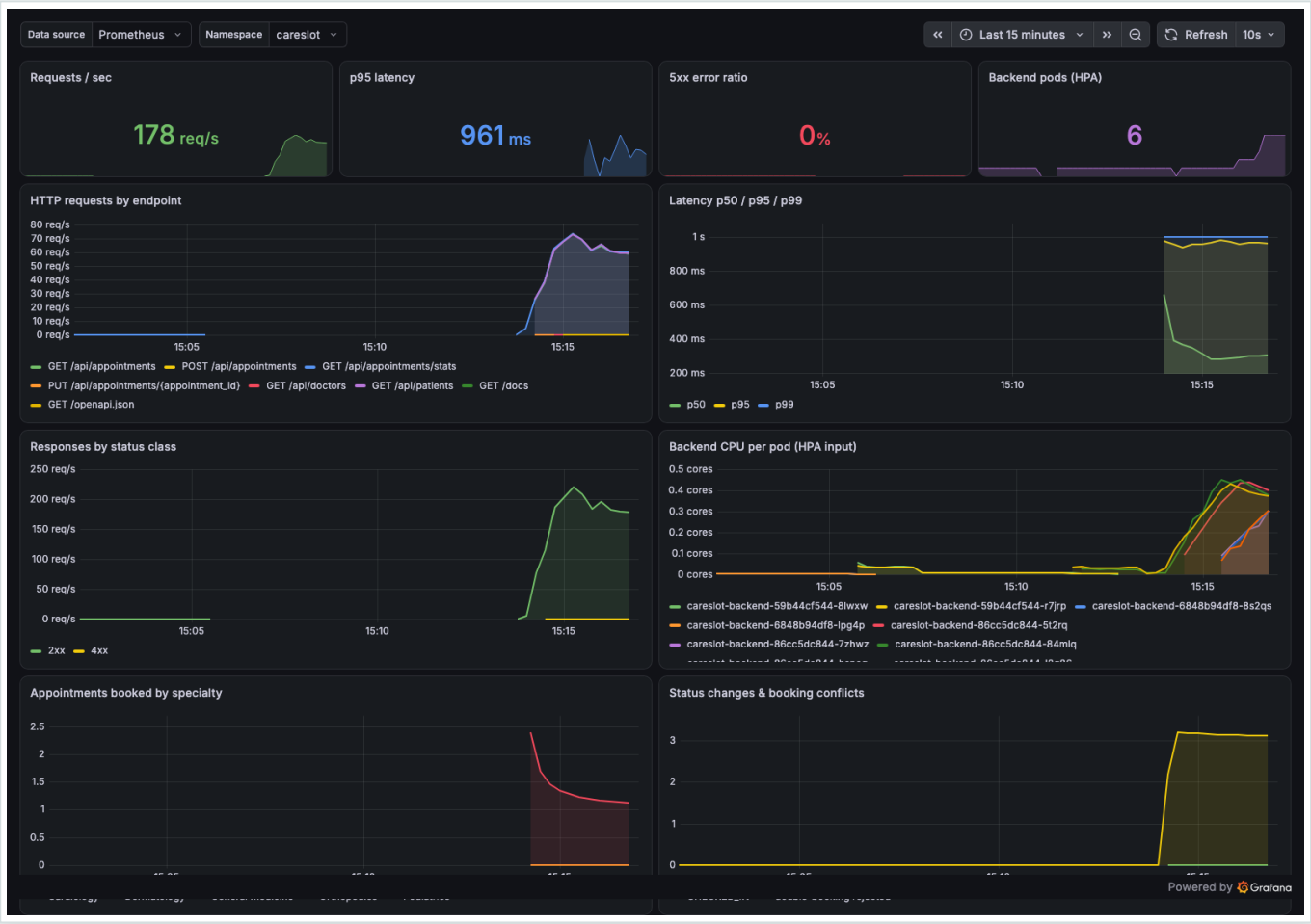
```
$ kubectl get hpa careslot-backend -n careslot   # during scripts/load-test.sh (60 workers)
NAME              REFERENCE                    TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
careslot-backend   Deployment/careslot-backend   cpu: 417%/60%   2         6         6          11m

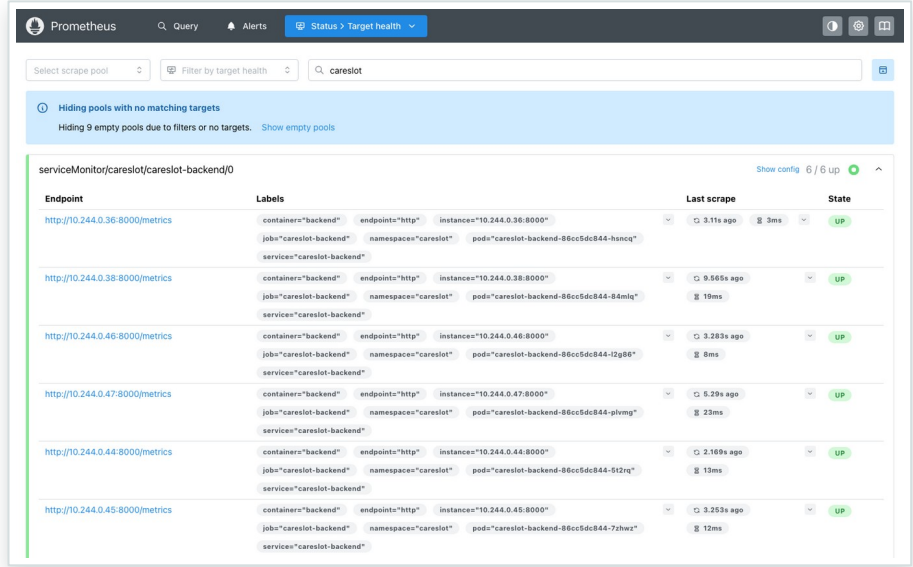
$ kubectl describe hpa careslot-backend -n careslot | grep -A6 Events
Events:
  Type     Reason                        Age                 From                       Message
  ----     ------                        ----                ----                       -------
  Normal   SuccessfulRescale             11m                 horizontal-pod-autoscaler  New size: 2; reason: Current number of replicas below Spec.MinReplicas
  Warning  FailedGetResourceMetric       4m19s (x6 over 11m)  horizontal-pod-autoscaler  failed to get cpu utilization: unable to get metrics for resource cpu:
  Warning  FailedComputeMetricsReplicas  4m19s (x6 over 11m)  horizontal-pod-autoscaler  invalid metrics (1 invalid out of 1), first error is: failed to get cpu
  Warning  FailedGetResourceMetric       3m34s (x7 over 10m)  horizontal-pod-autoscaler  failed to get cpu utilization: did not receive metrics for targeted pod
  Warning  FailedComputeMetricsReplicas  3m34s (x7 over 10m)  horizontal-pod-autoscaler  invalid metrics (1 invalid out of 1), first error is: failed to get cpu
  Normal   SuccessfulRescale             2m4s                horizontal-pod-autoscaler  New size: 3; reason: cpu resource utilization (percentage of request) a

$ kubectl get pods -n careslot -l app.kubernetes.io/component=backend
NAME                               READY   STATUS    RESTARTS   AGE
careslot-backend-86cc5dc844-5t2rq   1/1     Running   0          2m4s
careslot-backend-86cc5dc844-7zhwz   1/1     Running   0          64s
careslot-backend-86cc5dc844-84mlq   1/1     Running   0          4m47s
careslot-backend-86cc5dc844-hsncq   1/1     Running   0          5m
careslot-backend-86cc5dc844-l2g86   1/1     Running   0          64s
careslot-backend-86cc5dc844-plvmg   1/1     Running   0          64s

$ kubectl top pods -n careslot
NAME                               CPU(cores)   MEMORY(bytes)
careslot-backend-86cc5dc844-5t2rq   440m         96Mi
careslot-backend-86cc5dc844-7zhwz   391m         93Mi
careslot-backend-86cc5dc844-84mlq   408m         97Mi
careslot-backend-86cc5dc844-hsncq   404m         95Mi
careslot-backend-86cc5dc844-l2g86   385m         91Mi
careslot-backend-86cc5dc844-plvmg   387m         92Mi
careslot-frontend-5c65455d6-wcz9r   1m           23Mi
careslot-frontend-5c65455d6-zp9q6   1m           23Mi
careslot-postgres-0                 383m         170Mi
```

Observability: Prometheus scrapes, Grafana explains





- ServiceMonitor → 6/6 backend targets UP
- HTTP rate, p50/p95/p99 latency, 5xx ratio, CPU per pod
- Business metrics: bookings by specialty, status changes, rejected double-bookings
- Dashboard ships in the Helm chart (ConfigMap + Grafana sidecar)

Key learnings

Scanners find real problems

Trivy failed the very first image on a DoS CVE. The fix was a dependency bump plus a smaller runtime image.

Starting order is a deployment concern

Two replicas racing Alembic crash-looped. An advisory lock and a wait-for-db init container fixed it.

GitOps makes the cluster boring

CI writes the SHA to Git; Argo CD does the rest, and kubectl drift is reverted automatically.

Starter code had real bugs

Ingress pointed at the wrong port, service names didn't match, the HCL didn't parse. Troubleshooting is the job.

Thank you

Live walkthrough next: code → CI run → Terraform → Argo CD → kubectl → Prometheus → Grafana



github.com/MadaraUchiha-tech/devsecops-python

ghcr.io/madarauchiha-tech/careslot-backend · careslot-frontend